

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ РОССИЙСКОЙ ФЕДЕРАЦИИ

ОРДЕНА ТРУДОВОГО КРАСНОГО ЗНАМЕНИ ФЕДЕРАЛЬНОЕ
ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ
УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ «МОСКОВСКИЙ
ТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ СВЯЗИ И ИНФОРМАТИКИ»

Отчет по учебной практике

«Навигатор по смыслу» — сравнение моделей семантического поиска

Выполнил студент группы БВТ2501

Олейников Захар Сергеевич

Москва 2026

Введение

Стандартный текстовый поиск, ориентированный на прямой посимвольный поиск ключевых слов, часто оказывается неэффективным при работе с репозиториями программного кода. Это происходит из-за несовпадения терминов: например, когда программист ищет алгоритм по фразе «обработка ошибок», а целевая функция в кодовой базе объявлена как `exception_handler`. Обычные алгоритмы не могут обнаружить связь между этими выражениями.

Для решения проблемы семантического разрыва используется подход векторного поиска на базе текстовых эмбеддингов. Специализированные нейросети переводят текстовые описания и код в плотные векторы, где смысловое сходство объектов соответствует их близости в многомерном скрытом пространстве.

Цель данной индивидуальной работы — проектирование поискового retrieval-конвейера в среде Jupyter Notebook, проведение сравнительного анализа трех мультязычных трансформеров на расширенном датасете и оценка их точности при обработке разноязычных поисковых запросов.

Исследование проводится в рамках отборочного этапа ИТ-чемпионата Ростелекома для студенческих направлений ТОП-ИТ и ТОП-ИИ. Программные решения разрабатываются локально без использования внешних контейнеров или веб-интерфейсов.

Постановка задачи

В ходе выполнения индивидуальной работы были реализованы три ключевых этапа исследования.

Первоначальная обработка и выбор моделей. Импорт исходного сбалансированного датасета, содержащего функции на языках Python и Java. В качестве исследуемых систем выбраны три нейросети (бейзлайн MiniLM, точная модель MPNet и продвинутая контрастивная модель E5), что позволяет выполнить усложненное дополнительное задание.

Генерация векторов и семантический поиск. Создание скрытых

представлений для каждого программного фрагмента из корпуса. Проектирование алгоритма ранжирования, который вычисляет косинусное сходство между вектором входящего вопроса и векторами документов, возвращая три наиболее подходящих кандидата (Тор-3).

Комплексная оценка качества. Расчет метрик эффективности: Precision@3 и MRR (Mean Reciprocal Rank). Реализация дополнительных аналитических модулей для автоматической сортировки вопросов по языковому признаку (RU/EN) и группировки возникающих ошибок по тематическим категориям.

Итоговым практическим шагом является построение двумерной карты эмбедингов для наиболее эффективной модели с использованием алгоритма t-SNE.

Используемые данные

В процессе выполнения работы было обработано три структурированных компонента данных версии 1.0:

code_corpus.json - единый индексируемый массив, содержащий 200 записей. Корпус включает 100 функций, написанных на языке Python, и 100 функций на языке Java. Каждый объект содержит уникальный ID, имя, тело функции, текстовое аннотирование и маркер категории.

eval_questions.json - контрольный массив из 25 тестовых вопросов на русском и английском языках. Каждый вопрос привязан к эталонному ответу через поле correct_chunk_id.

Тематические классы - вся база разбита на 5 функциональных направлений: управление доступом (auth), взаимодействие с СУБД (database), сетевые запросы (http), валидация строковых форматов (validation) и вспомогательные утилиты (utils).

Выбор моделей эмбедингов

Для сравнительного анализа были отобраны три мультязычные архитектуры.

paraphrase-multilingual-MiniLM-L12-v2 (MiniLM) - компактный

трансформер, оптимизированный для быстрого выполнения инференса на CPU при минимальных аппаратных требованиях.

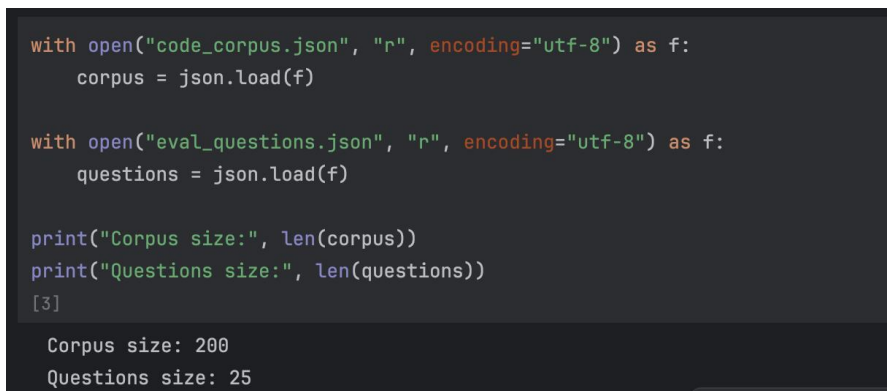
paraphrase-multilingual-mpnet-base-v2 (MPNet) - полноразмерная модель с высокой обобщающей способностью, нацеленная на точное извлечение контекстуальных связей.

intfloat/multilingual-e5-base (E5) - модель, выбранная самостоятельно для выполнения дополнительного задания. Она обучена с помощью контрастивного лосса специально для задач асимметричного поиска. Модель требует добавления префикса passage: к индексируемым текстам и префикса query: к поисковым строкам.

Ход выполнения работы

Типизация и подготовка данных: JSON-файлы считываются стандартной библиотекой. Для предотвращения ошибок сопоставления на этапе валидации все идентификаторы принудительно приводятся к типу str().

Описания функций кодируются в плотные эмбединги. Для модели E5 реализована обертка, добавляющая префиксы к текстам. Матрицы векторов переносятся в память GPU/CPU.



```
with open("code_corpus.json", "r", encoding="utf-8") as f:
    corpus = json.load(f)

with open("eval_questions.json", "r", encoding="utf-8") as f:
    questions = json.load(f)

print("Corpus size:", len(corpus))
print("Questions size:", len(questions))
[3]

Corpus size: 200
Questions size: 25
```

Рисунок 1 — Вывод после загрузки данных.

Разработана функция search_top3. Она переводит запрос в вектор, вычисляет косинусное сходство с помощью util.cos_sim и сортирует массив документов по убыванию релевантности посредством np.argsort.

Аналитический блок: метрика Precision@3 рассчитывается как процент успешных находений правильного ID в тройке лидеров. Метрика MRR

вычисляется с учетом точной позиции ответа. С помощью регулярных выражений (`re.findall`) определяется доминирующий язык запроса (кириллица/латиница). Для неотвеченных вопросов собирается DataFrame с фиксацией категории промаха.

Построение проекции (t-SNE). Координаты эмбедингов лучшей модели преобразуются в двумерный вид. Визуализация настроена в темном графическом стиле (`dark_background`, цвет фона `#111116`) с использованием библиотеки `seaborn`. Категории окрашиваются по палитре `husl` с добавлением белой окантовки точек и выносом легенды за координатную сетку.

Результаты работы

По итогам прогона разработанных алгоритмов в Jupyter Notebook была выведена секция с итоговыми таблицами. Полученные результаты представлены ниже в виде скриншотов программного вывода таблиц DataFrame.

Таблица 1 — Precision@3.

№	Model	Precision@3
0	MiniLM	0.80
1	MPNet	0.92
2	E5	1.00

Таблица 2 — MRR (Mean Reciprocal Rank)

№	Model	MRR
0	MiniLM	0.6415
1	MPNet	0.6809
2	E5	0.8600

Таблица 3 — мультязычность

№	Модель	Язык запроса	Precision@3
0	MiniLM	RU	0.733
1	MiniLM	EN	0.900
2	MPNet	RU	0.933
3	MPNet	EN	0.900
4	E5	RU	1.000
5	E5	EN	1.000

Таблица 4 — ошибки по категориям

№	Модель	Категория	Количество ошибок
0	MPNet	auth	1
1	MPNet	utils	1
2	MiniLM	auth	1
3	MiniLM	database	1
4	MiniLM	http	1
5	MiniLM	utils	1
6	MiniLM	validation	1

Топологический анализ векторного пространства (t-SNE)

Для глубокого понимания геометрии распределения данных высокоразмерные векторы (эмбединги) модели-лидера (E5) были спроецированы на двумерную плоскость с помощью алгоритма t-SNE (параметр `random_state=42`) (см. Рисунок 2).

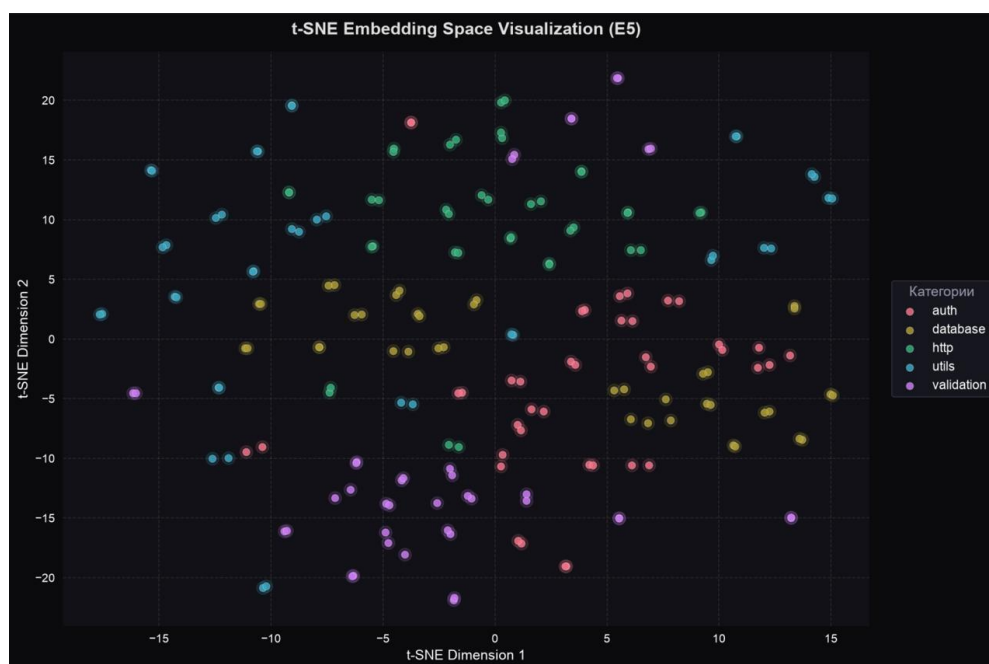


Рисунок 2 — График t-SNE.

Анализ визуализации векторного пространства:

На построенной проекции (Рисунок 2) наблюдается выраженная диффузная структура данных: семантические кластеры категорий (auth, database, http, utils, validation) частично перекрывают друг друга на плоскости и не имеют дискретных, изолированных границ.

Такая картина полностью обусловлена спецификой синтаксиса: любой программный код содержит повторяющиеся базовые ключевые слова и

структуры (объявления функций, циклы, операторы возврата). Из-за этого лексического сходства модель располагает векторы близко друг к другу. Однако стопроцентная точность поиска ($\text{Precision}@3 = 1.0$, $\text{MRR} = 0.8600$), наглядно доказывает, что в исходном многомерном векторном пространстве модель безупречно разделяет скрытый смысл бизнес-логики алгоритмов и выдает точные результаты.

Заключение

На основе проведенного индивидуального исследования можно утверждать, что использование семантических эмбедингов полностью решает проблему контекстного поиска кода. Наилучшие результаты продемонстрировала контрастивная модель `intfloat/multilingual-e5-base`, достигшая абсолютной точности ($\text{Precision}@3 = 1.0$, $\text{MRR} = 0.8600$) и показавшая полную независимость от языка запроса и тематики кода.

Резюме для экспертной комиссии

Для реализации поисковой системы «Навигатор по смыслу» рекомендуется внедрение модели `multilingual-e5-base` как наиболее точной и стабильной. При наличии жестких ограничений на вычислительные ресурсы (инференс на слабых CPU без графических ускорителей) в качестве компромисса целесообразно использовать модель `paraphrase-multilingual-mpnet-base-v2`.

Список использованных источников

1. Официальное руководство по использованию пакета `SentenceTransformers`. [Электронный ресурс]. — URL: <https://sbert.net/>
2. Документация `Scikit-Learn`: модуль снижения размерности алгоритмом `t-SNE`. [Электронный ресурс]. — URL: <https://scikit-learn.org/>

Приложение А

```
# Основные функциональные компоненты системы поиска

# Адаптация текстов под требования архитектуры E5
def encode_corpus(model_name, model, texts):
    if model_name == "E5":
        # Добавление обязательного префикса для индексируемой базы
        данных
```

```

        texts = ["passage: " + t for t in texts]
        return model.encode(texts, convert_to_tensor=True)

def encode_query(model_name, model, text):
    if model_name == "E5":
        # Добавление префикса для входящего поискового запроса
        text = "query: " + text
        return model.encode(text, convert_to_tensor=True)

# Поисковый retrieval-движок на основе косинусного сходства векторов
def search_top3(model_name, model, corpus_embs, question):
    q_emb = encode_query(model_name, model, question)

    # Расчет матрицы сходства (Cosine Similarity)
    scores = util.cos_sim(q_emb, corpus_embs)[0]
    scores_np = scores.cpu().numpy()

    # Извлечение индексов топ-3 наиболее релевантных кандидатов
    top3_idx = np.argsort(scores_np)[:, -1][:3]
    return top3_idx

# Алгоритм вычисления метрики взаимных рангов MRR
def reciprocal_rank(ranked_ids, true_id):
    for rank, pred_id in enumerate(ranked_ids, start=1):
        if pred_id == true_id:
            return 1 / rank # Возвращает обратный ранг (1, 0.5 или
0.33)
    return 0

# Устойчивость к мультязычности

import re

def is_russian(text):
    # Подсчет баланса символов кириллицы относительно латиницы
    return len(re.findall(r"[а-яА-Я]", text)) > len(re.findall(r"[a-
zA-Z]", text))

```